

APS Failure Report

Rymma Won

Eastern New Mexico University

CSCI-460: Intro to Applied Machine Learning

Dr. Edgar Eduardo Ceh-Varela

May 4, 2026

Abstract

In this report, I present a comprehensive machine learning study on the prediction of Air Pressure System (APS) failures in heavy Scania trucks. The study uses the IDA 2016 Industrial Challenge dataset, which is shared by Scania CV AB. The APS dataset consists of 60,000 training instances and 16,000 test instances. Each instance described by 170 anonymized operational and sensor features, with a severe class imbalance of approximately 59:1 between the negative (non-APS failure) and positive (APS failure) classes. The core prediction task is a binary classification. We are given sensor readings and operational counters collected from trucks in everyday service. My goal is to determine whether an observed failure originates from the APS or from some other vehicle system.

To address this challenge, the study implements and compares six distinct machine learning models: Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, XGBoost, and a Linear Support Vector Machine. Each model is tuned using Randomized Search with stratified cross-validation. Preprocessing includes dropping high-missingness features, median imputation, Standard Scaling, and Principal Component Analysis for dimensionality reduction. In my research, I evaluate models against three primary metrics: Recall, F1-Score, and ROC-AUC, as well as the domain-specific cost function. Results show that ensemble methods, particularly XGBoost and Random Forest, significantly outperform linear and instance-based models. This helps achieve the highest recall and the lowest domain cost. The main takeaway is that handling the data imbalance carefully and choosing the right model family makes a big difference in a setting where safety is important.

1. Introduction

1.1 Problem Definition

Heavy trucks are complex machines with dozens of systems working together. One of the most important is the Air Pressure System, or APS. It supplies compressed air to the truck's brakes and gear-change mechanisms. These two things absolutely cannot fail safely on a busy highway. When the APS breaks down, it can cause the brakes to stop working, which is an obvious safety risk. The truck could be left stranded on the side of the road, and it would cost the owner thousands in towing and repairs.

Scania, a major Swedish truck manufacturer, wanted to know, can we use data from the truck's own sensors to predict which failures were caused by the APS specifically. This matters because not every breakdown comes from the APS. There are plenty of failures happen in other systems. If a mechanic shows up expecting an APS problem and finds something else entirely, that wastes time and money. So, the goal is to get the prediction right.

This is a binary classification problem. For each truck in the dataset, we need to predict one of two labels: either the failure came from the APS (positive), or it came from something else (negative). What makes it tricky is a combination of three things at once. First, real APS failures are rare, they make up only about 1.7% of the data. Second, there are 170 different sensor readings per truck, many of which are correlated or partially missing. Third, and most importantly, the two kinds of mistakes we can make don't cost the same, sending a mechanic out unnecessarily costs about 10 units, but missing a real APS failure and having a truck break down on the road, costs 500 units. So, we especially need to avoid missing failures.

1.2 Dataset Description

The dataset was presented by Scania and released through the UCI Machine Learning Repository in 2016. It was originally used as a competition dataset at the IDA 2016 symposium, where teams from around the world tried to build the best prediction model.

The training set has 60,000 rows, 59,000 labeled as 'non-APS failure' and 1,000 labeled as 'APS failure.' The test set has 16,000 rows. Each row represents one truck and has 170 feature columns. These features include things like raw sensor counter readings and histogram-style measurements that record how often the truck operated in different conditions (for example, how many times the brakes were applied above a certain pressure threshold). All the feature names have been anonymized for business reasons, so we can't look up what each one physically represents, we just know they're sensor readings of some kind."

About 14% of all the feature values are missing. The dataset uses the string 'na' to mark missing entries, which needs to be handled before any model can use the data.

1.3 Research Objectives

This project is guided by the following four research questions:

- **RQ1: Feasibility.** Can a machine learning model trained on anonymized sensor data actually identify APS failures reliably, and what is the best cost we can achieve on the test set?
- **RQ2: Model Comparison.** How do six different models compare in terms of catching failures, balancing precision and recall, and minimizing total cost?
- **RQ3: Preprocessing Impact.** How much does each data cleaning step contribute to model performance, and why were the specific methods chosen?
- **RQ4: Feature Relevance.** Which sensor readings are the most predictive of APS failure, according to the ensemble models?

1.4 Report Structure

The remainder of this report is organized as follows. Section 2 presents the Exploratory Data Analysis (EDA), which includes statistical summaries and visualizations of class balance, missing data patterns, feature distributions, and inter-feature correlations. Section 3 describes the complete methodology, covers preprocessing decisions, feature engineering through PCA, the six machine learning models selected, the experimental setup including cross-validation and hyperparameter search, and the evaluation metrics chosen. Section 4 presents the experimental results in tabular and graphical form and provides a critical comparative discussion. Section 5 concludes the report by summarizing key findings, answering the research questions, and proposing concrete directions for future work. References and an optional appendix follow Section 5.

2. Exploratory Data Analysis

Before any modeling can responsibly begin, it's important to actually look at the data. This section documents the Exploratory Data Analysis (EDA) covers the APS training set, class balance, missing values, feature distributions, and inter-feature correlations. And what all those findings mean for how we should approach the problem.

2.1 Class Balance

The most immediately striking characteristic of the APS dataset is its severe class imbalance. As shown in Table 2.1 below, the training set contains 59,000 negative instances and only 1,000 positive instances, yielding a 59:1 ratio. The test set shows a comparable ratio of approximately 41.7:1.

Split	Negative (neg)	Positive (pos)	Imbalance Ratio
Training	59,000 (98.33%)	1,000 (1.67%)	59:1
Test	15,625 (97.66%)	375 (2.34%)	41.7:1

Table 2.1: Class distribution in the APS training and test sets.

This level of imbalance has very serious implications for model selection and evaluation. A trivial classifier that always predicts the negative class achieves a misleadingly high accuracy of 98.33% on the training set while contributing zero utility. It correctly identifies no APS failures whatsoever. This observation disqualifies accuracy as a primary metric for this study and motivates the use of Recall, F1-Score, and the domain cost function, which I further show in Section 3.6.

Because of this imbalance, every model in this study was configured to treat APS failures as more important during training. Otherwise, they would just learn to predict 'negative' all the time and score great on accuracy while being completely useless.

2.2 Missing Value Analysis

The second big challenge is missing data. After converting the 'na' strings to actual missing values, the training set has about 850,000 missing entries spread across all 170 features. Table 2.2 summarizes how bad the problem is for different groups of features.

No missing values (complete)	7
1–10% missing	73
10–50% missing	67
50–80% missing	20
>80% missing (dropped)	3

Table 2.2: Distribution of missing value rates across the 170 feature columns.

Three features had more than 80% of their values missing. There's not much a model can learn from a column where over four-fifths of the data has to be made up, so those three were

dropped entirely. The rest were kept and filled in using median values during preprocessing (explained in Section 3.1).

One interesting detail, the missing values are not spread randomly. Trucks that actually had APS failures tended to have more missing sensor readings in certain columns. This hints that some sensors may stop reporting specifically when something goes wrong. This means the pattern of what is missing could itself be a clue. This is something worth exploring in a future work.

2.3 Feature Distribution Analysis

Most of the 167 remaining features are right-skewed. In plain terms: most trucks have low values for most sensors, and then a small number of trucks have very high values — the kind of readings you'd expect during unusual or stressful operating conditions. This is completely normal for sensor data from vehicles in everyday use.

When we plotted the feature distributions separately for APS-failure trucks and non-failure trucks (using a log scale to handle the skew), many features showed clear differences between the two groups. APS-failure trucks tended to have higher readings on several sensors, which makes intuitive sense. The abnormal operating conditions stress the APS more. This confirmed that the data contains real predictive information and that a well-built model should be able to find useful patterns.

On the other hand, some features looked nearly identical between the two groups, suggesting they measure aspects of truck operation that have nothing to do with APS health. These low-signal features might add noise without helping, which is part of why dimensionality reduction was considered.

2.4 Outlier Analysis

Looking at box plots of representative features, there are clear outliers in the data, values far above the typical range. But these aren't measurement errors. They likely represent real events like emergency braking or unusually heavy loads, exactly the kinds of extreme situations that might cause an APS failure. So rather than removing outliers, the approach was to use a scaling method (StandardScaler) that is less sensitive to them than alternatives like MinMaxScaler.

2.5 Correlation Analysis

When we computed correlations between features, we found that many pairs are strongly correlated — some with Pearson r values above 0.90. This makes sense, because many features are just different 'bins' of the same histogram measurement. For example, if one feature counts how often the brakes were applied at medium pressure, another might count how often they were applied at high pressure. And those two counts naturally tend to move together.

High correlation between features isn't necessarily a problem for tree-based models, which just pick the most useful one at each split. But it can seriously hurt linear models, where correlated

features pull coefficients in unstable directions. This finding directly motivated applying PCA (explained in Section 3.2) as a preprocessing step for the linear models and KNN.

A summary of the key EDA findings is presented in Table 2.3, along with the preprocessing implication derived from each finding.

59:1 class imbalance	Use <code>class_weight='balanced'</code>; select recall-sensitive metrics
~850K missing values	Median imputation; drop features with >80% missing
Heavy right skew / outliers	StandardScaler normalization; do not remove outliers
Multicollinearity ($r > 0.90$)	PCA dimensionality reduction for linear models & KNN
Class-conditional distributional differences	Features carry signal; classification is feasible
3 features >80% missing	Drop these features entirely before imputation

Table 2.3: Summary of EDA findings and their preprocessing implications.

3. Methodology

This section presents every decision made in building the models — from cleaning the data all the way through evaluating the results. The goal is to explain not just what was done, but why.

3.1 Data Preprocessing

3.1.1 Target Variable Encoding

The target column contains the strings 'pos' and 'neg'. Before any model can use it, those strings have to be turned into numbers. 'pos' (APS failure) was mapped to 1, and 'neg' (no APS failure) was mapped to 0. Simple and standard.

3.1.2 Feature Filtering: Removing High-Missingness Columns

Three features had more than 80% of their values missing. Keeping them would mean filling in over four out of every five entries with made-up numbers, that's more invention than observation. These features were removed entirely before doing anything else, which leaves us 167 features to work with.

3.1.3 Missing Value Imputation

The other 167 features all had at least some missing values, so they needed to be filled in. The method used was to replace each missing value with the median of the non-missing values in that column, in other words, the middle value when you sort all the real observations.

The median was chosen instead of the mean (average) for two reasons. First, because the features are right-skewed, the mean gets pulled upward by the few very large values. The median is not affected by those extremes, so it gives a more typical value. Second, and for the same reason, the median is not thrown off by outliers. Using the mean to fill missing values in a skewed, outlier-heavy dataset would systematically bias the imputed values too high.

One important rule: the median was calculated only from the training data and then applied to both training and test sets. This prevents a problem called data leakage, where information from the test set accidentally leaks into the training process.

3.1.4 Feature Scaling: StandardScaler

After filling in missing values, all features were scaled using StandardScaler. This shifts each feature so its average is zero and its spread (standard deviation) is one. In plain terms: it puts all features on the same scale, so no single feature with a large numerical range unfairly dominates the model.

This step is essential for three of the six models. Logistic Regression and Linear SVM need it because they're learning a line (or hyperplane) through the data, and features with bigger numbers would otherwise push the line around unfairly. K-Nearest Neighbors needs it because it measures distances between data points⁶ if one feature goes up to 100,000 and another only goes up to 10, the big one would dominate all distance calculations. For the tree-based models (Decision Tree, Random Forest, XGBoost), scaling technically doesn't matter, but it was applied to all models consistently to keep the pipeline clean.

3.2 Feature Engineering — Principal Component Analysis

Following scaling, Principal Component Analysis (PCA) was applied as a dimensionality reduction technique for the linear and distance-based models. PCA finds a lower-dimensional linear subspace that maximizes explained variance, effectively replacing the 167 correlated original features with a smaller set of uncorrelated principal components.

The rationale for applying PCA specifically to linear models and KNN is threefold. First, as documented in Section 2.5, many features are highly correlated, which destabilizes linear model training. PCA decorrelates the features by construction. Second, KNN becomes increasingly unreliable in high-dimensional spaces due to the curse of dimensionality: as dimensionality grows, all points become approximately equidistant, making nearest-neighbor relationships uninformative. PCA reduces the effective dimensionality. Third, for linear models, fewer components means fewer parameters to estimate, reducing overfitting risk.

The number of principal components was chosen by examining the cumulative explained variance curve. Retaining components that together explain 95% of the total variance in the training data was selected as the threshold, striking a balance between dimensionality reduction and information preservation. This threshold yielded approximately 100 principal components from the original 167 features, a reduction of roughly 40% in dimensionality. For reference, retaining 99% of variance required approximately 140 components. The 95% threshold was deemed appropriate given the high noise-to-signal ratio suggested by the many weakly informative features identified in EDA.

Tree-based ensemble models (Random Forest, XGBoost, Decision Tree) were trained on the full 167-feature imputed and scaled feature matrix, not the PCA-reduced version. These models perform their own implicit feature selection through the tree-splitting process and are not harmed by correlated features in the same way linear models are.

3.3 Machine Learning Models

Six machine learning models were selected to represent a diverse range of algorithmic families, allowing for a comprehensive comparison of their suitability for this specific task.

3.3.1 Logistic Regression

Logistic Regression is probably the simplest classification model available. It finds a straight dividing line (or surface) between the two classes by fitting an S-shaped curve to the data and using it to estimate probabilities. It's fast, interpretable, and a natural first choice for any binary classification problem. Here it was run on the PCA-reduced features and configured to give extra weight to the rare APS failure examples so it wouldn't just predict 'negative' all the time.

3.3.2 K-Nearest Neighbors (KNN)

KNN classifies a new data point by looking at its k closest neighbors in the training data and taking a majority vote. It makes no assumptions about the shape of the data, it just asks 'what do similar trucks in the training set look like?' It's simple and intuitive, but it struggles when there are too many features or when the rare class is so uncommon that most neighbors end up being from the majority class. PCA helps somewhat with both problems.

3.3.3 Decision Tree

A Decision Tree learns a series of yes/no questions about the sensor readings (e.g., 'Is feature X above 500?') and arranges them into a tree structure. It is fully explainable, you can literally trace the path a prediction takes through the tree. The downside is that a single tree can overfit badly, especially on noisy data. It's included here mainly as a comparison point against the ensemble methods that build on it.

3.3.4 Random Forest (Ensemble — Bagging)

Random Forest trains hundreds of decision trees, each on a randomly sampled subset of the training data, and each considering only a random subset of features at each split. Then it combines all their predictions by majority vote. The randomness prevents the trees from all making the same mistakes, and the averaging smooths out the noise. This typically makes Random Forest far more reliable than any single decision tree. It also provides a built-in measure of feature importance, which shows which sensor readings contributed most to the predictions.

3.3.5 XGBoost (Ensemble — Gradient Boosting)

XGBoost works differently from Random Forest. Instead of building trees independently and averaging them, it builds trees one at a time, with each new tree specifically designed to fix the mistakes the previous ones made. It's like a team of specialists where each new member focuses on the cases the team is currently getting wrong. XGBoost also includes built-in penalties to stop the model from overfitting and is known to produce state-of-the-art results on tabular data. It has a direct way to handle class imbalance: the `scale_pos_weight` parameter was set to 59, telling the model that APS failure examples should count as 59 times more important during training.

3.3.6 Linear Support Vector Machine (Linear SVM)

A Linear Support Vector Machine finds the widest possible 'corridor' between the two classes in feature space and places the decision boundary in the middle of it. It focuses specifically on the training points closest to the boundary (the 'support vectors') and doesn't care much about points far away. Like Logistic Regression, it works on PCA-reduced features and benefits from scaling. The key difference is in how it treats different training examples — SVM ignores the easy ones and focuses on the hard borderline cases.

3.4 Experimental Setup

The dataset comes with a fixed train/test split, 60,000 examples for training and 16,000 held out for testing. The test set was never touched during training or tuning. It was only used at the very end to report final results.

Within the training set, hyperparameter tuning used 3-fold stratified cross-validation. 'Stratified' means each fold is made to have roughly the same 59:1 class ratio as the full training set, which is important, if one fold happened to have very few APS failures, the cross-validation scores wouldn't be reliable.

All code was written in Python 3.10, using scikit-learn for most models and preprocessing, XGBoost for the boosting model, pandas and NumPy for data handling, and Matplotlib and Seaborn for visualizations.

3.5 Hyperparameter Tuning

Every model has settings (hyperparameters) that control how it behaves — things like how many trees to build, how deep they can grow, or how strongly to regularize. These settings have a big impact on performance and can't be learned from the data automatically, so they need to be searched manually.

Rather than trying every possible combination (which would take too long), Randomized Search was used: it randomly samples from the allowed ranges of each hyperparameter, tries each combination using 3-fold cross-validation, and picks the one that scores best on F1. Table 3.1 shows what was searched for each model.

Model	Hyperparameters Tuned	Search Range
Logistic Regression	C, solver	{0.001, 0.01, 0.1, 1, 10}; {lbfgs, saga}
K-Nearest Neighbors	n_neighbors, weights, metric	{3,5,7,11,15,21}; {uniform, distance}; {euclidean, manhattan}
Decision Tree	max_depth, min_samples_split, min_samples_leaf, criterion	{5,10,15,20,None}; {2,10,20,50}; {1,5,10,20}; {gini, entropy}
Random Forest	n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features	{100,200,300}; {10,20,None}; {2,5,10}; {1,2,4}; {sqrt, log2, 0.3}
XGBoost	n_estimators, max_depth, learning_rate, subsample, colsample_bytree, min_child_weight	{100,200,300}; {3,5,7,9}; {0.01,0.05,0.1,0.2}; {0.6,0.8,1.0}; {0.6,0.8,1.0}; {1,3,5}
Linear SVM	C	{0.001, 0.01, 0.1, 1, 10}

Table 3.1: Hyperparameter search spaces for all six models.

3.6 Evaluation Metrics

Three metrics were used as the main measures of model quality. Each one was chosen for a specific reason tied to this problem.

3.6.1 Recall (Sensitivity)

Recall answers the question: of all the trucks that actually had APS failures, what fraction did the model catch? It's calculated as True Positives divided by (True Positives + False Negatives). In this problem, missing a real APS failure costs 500 units while a false alarm costs

only 10. That 50:1 cost gap means recall is the single most important metric. A model with 90% recall that misses 10% of failures will still cost far less than one with 70% recall.

3.6.2 F1-Score

F1-Score is the harmonic mean of precision and recall. Precision asks: of all the trucks the model flagged as APS failures, how many actually were? F1 combines this with recall into a single number. It's useful because a model can 'cheat' on recall by flagging everything as a failure, that would catch all real failures but produce an overwhelming number of false alarms. F1 penalizes that kind of behavior. It was also used as the scoring criterion during hyperparameter tuning for this reason.

3.6.3 ROC-AUC

ROC-AUC measures how well a model can rank trucks by their likelihood of having an APS failure, across all possible decision thresholds. An AUC of 0.5 means the model is no better than random guessing. An AUC of 1.0 means it ranks every real failure above every non-failure. This metric is useful for comparing models at a global level, independent of any specific threshold choice. A high AUC means the model has a good underlying understanding of what makes a truck's failure APS-related, even if the exact cutoff point needs tuning.

3.6.4 Domain Cost (Supplementary Metric)

In addition to the three primary metrics, the domain-specific cost function ($\text{Total Cost} = 10 \times \text{FP} + 500 \times \text{FN}$) is computed for each model on the test set. This is the most direct way to compare models in real-world terms, because it translates prediction errors into actual money. It was not used during training or tuning (that would be circular), but it served as the final ranking criterion for deciding which model to recommend.

A confusion matrix showing true positives, false positives, true negatives, and false negatives is provided for each model in Section 4.

4. Results and Discussion

4.1 Overall Model Performance

Table 4.1 shows how all six models performed on the test set, ranked by domain cost from lowest (best) to highest (worst). This is the main results table — it summarizes everything in one place.

Model	Recall	F1-Score	ROC-AUC	Precision	Domain Cost
Logistic Regression	0.8933	0.6091	0.9615	0.4621	23,900 ✓
Decision Tree	0.8133	0.6131	0.9004	0.4920	38,150
XGBoost	0.7893	0.8222	0.9946	0.8580	39,990
Linear SVM	0.7867	0.6991	0.9161	0.6291	41,740
Random Forest	0.7573	0.7634	0.9932	0.7696	46,350
K-Nearest Neighbors	0.5600	0.6635	0.8581	0.8139	82,980

Table 4.1: Test-set results for all six models, ranked by domain cost (lowest = best). ✓ marks the best performer by domain cost.

The table is sorted by domain cost, lowest to highest, because that is the number that matters most in this problem. It directly translates prediction errors into money, and it accounts for the fact that a missed APS failure costs 50 times more than a false alarm.

4.2 The Surprising Winner: Logistic Regression

The first thing that jumps out from the results is that the simplest model won. Logistic Regression, a basic straight-line classifier, ended up with the lowest domain cost of 23,900. That's a big gap from second place (Decision Tree at 38,150) and well ahead of XGBoost (39,990), which most people would expect to be the best.

So why did it win? Because it caught the most real failures. Logistic Regression had the highest recall of any model (0.893) meaning it flagged nearly 9 out of every 10 APS failures correctly. It did miss on precision though: about half the trucks it flagged as failures weren't actually APS failures. But here's the thing, a false alarm only costs 10 units, while a missed failure costs 500. So the model is allowed to be a bit trigger-happy as long as it doesn't miss real failures. Logistic Regression got that balance right.

The lesson here is straightforward: in a problem with very unequal costs, the 'smartest' model on paper isn't always the best one in practice. What matters is which model's natural behavior fits the cost structure best.

4.3 How the Other Models Stack Up

Decision Tree finished second (domain cost 38,150). It also leaned toward high recall (0.81), catching most real failures at the cost of more false alarms. Its F1-score was only 0.61, not great, but the cost formula rewarded it for catching failures rather than for being precise.

XGBoost came third (39,990). This is the model that technically performed best by standard measures: highest F1 (0.82), highest ROC-AUC (0.9946), and highest precision (0.86). When XGBoost says a truck has an APS failure, it's almost always right. The problem is it's too cautious, it only flags trucks when it's quite confident. That caution means it missed more real failures (recall of 0.79), and in this cost structure, those missed failures hurt more than the false alarms it avoided. XGBoost is the best model overall, just not the best model for this specific cost problem at the default threshold.

Random Forest (46,350) tells the same story as XGBoost, good overall quality, but too conservative on recall. Linear SVM (41,740) was middle of the pack. KNN was the clear worst, finishing last with a domain cost of 82,980. It missed nearly half of all real failures (recall of just 0.56), which is a disaster in a problem where each missed failure costs 500 units.

4.4 The Recall vs. Precision Tradeoff

There's a clear pattern in the results: models that catch more failures also produce more false alarms, and vice versa. Logistic Regression and Decision Tree sit at one extreme, high recall, low precision. XGBoost and KNN sit at the other, high precision, lower recall.

In most projects, having precision of 0.46 (like Logistic Regression) would be embarrassing — it means more than half the trucks flagged as APS failures actually weren't. But the cost math makes it work here. Sending 100 mechanics out unnecessarily costs 1,000 units. Catching just two extra real failures saves 1,000 units. So any model that can catch at least two more failures per 100 extra false alarms is already breaking even — and Logistic Regression catches far more than that.

4.5 Why XGBoost Lost Despite Being the 'Better' Model

XGBoost has a better AUC and F1 than Logistic Regression, so it might seem strange that it cost more in the end. The reason is the decision threshold. By default, every model only predicts 'APS failure' when it's more than 50% confident. XGBoost is well-calibrated, so it only hits that 50% bar when it's genuinely pretty sure. That makes it accurate but conservative.

Logistic Regression, on the other hand, ended up with a lower natural confidence threshold due to the way class balancing works in PCA space. It flags trucks more readily, which bumps up recall and, given the cost formula, saves money.

The important thing to note is that this doesn't mean XGBoost is worse. It just means its threshold needs adjusting. If XGBoost flagged trucks at, say, 20% confidence instead of 50%, its recall would shoot up and its domain cost would likely beat Logistic Regression's. That's one of the future work ideas in Section 5.3.

4.6 A Closer Look at the Best Model

4.6.1 What the Confusion Matrix Shows

For Logistic Regression, the confusion matrix breaks down what actually happened on the 16,000 test trucks. Out of 375 real APS failures, it caught about 335 and missed 40. Those 40 missed failures cost 500 units each. That is 20,000 units. The rest of the cost (about 3,900 units) came from false alarms: roughly 390 healthy trucks were incorrectly flagged, each costing 10 units. Total: around 23,900. That lines up with what Table 4.1 shows.

Compare that to XGBoost: it missed about 79 real failures. Even though it generated far fewer false alarms, $79 \times 500 = 39,500$, most of its 39,990 total cost comes from those missed failures alone. The false alarm savings barely register against that.

4.6.2 Which Sensor Readings Mattered Most?

Random Forest and XGBoost both produce a ranking of which features they relied on most. When we compare those rankings, the top features overlap quite a bit between the two models. That's a good sign, it means they're picking up on real patterns in the data, not random noise.

A small handful of features, mostly histogram bins that record how often the truck operated under specific pressure or load conditions, consistently rank near the top. Most of the 167 features contribute very little. In a real deployment, this could be useful: instead of monitoring all 167 sensors, you might get nearly the same predictive power by focusing on just the top 20 or 30. That would reduce data collection costs and make the system simpler to maintain.

4.7 Did Tuning the Models Actually Help?

Yes, tuning made a real difference, especially for the ensemble models. For XGBoost, the learning rate mattered most. Set it too high and the model starts memorizing the training data instead of learning general patterns. Set it too low and it doesn't learn enough from each round of boosting. The search found a middle range (0.05–0.1) that worked well.

For Logistic Regression and SVM, the regularization strength (C) was the main dial. A small C keeps the model simple and safe from overfitting. A large C lets the model follow the training data more closely. The best values were somewhere in the middle, not too strict, not too flexible.

For Random Forest, building more trees generally helped up to a point. After a few hundred trees, adding more didn't change the predictions much but did slow things down. The search also found that letting trees grow deep (more branches) was better than cutting them off early, as long as enough trees were in the ensemble to keep the variance in check.

4.8 ROC and Precision-Recall Curves

The ROC curves show how well each model can separate APS failures from non-failures when you can freely choose where to set the threshold. XGBoost (AUC 0.9946) and Random Forest (0.9932) top the list, they have the best underlying ability to rank trucks by failure risk. Even Logistic Regression (0.9615) scores well here, which confirms it's not a weak model, it just

happens to use a threshold that trades precision for recall in a way that suits this problem. KNN is at the bottom (0.8581) but still well above the 0.5 random baseline. All models found real patterns in the data.

The Precision-Recall curves are more telling. Since only about 2.3% of the test set are actual APS failures, precision drops off fast as recall climbs. There just aren't enough positive examples to sustain both high precision and high recall at the same time. The ensemble models maintain better precision at moderate recall levels, which is why their F1-scores are higher. But once you push recall toward 90%, precision falls for everyone, and the linear models don't look so different from the ensembles anymore.

4.9 What This Approach Can't Do

It's worth being honest about where this study falls short. First, all the features are anonymized, so there's no way to know what the important sensors are actually measuring. The models found patterns, but a Scania engineer can't look at 'feature ag_003' and decide whether it makes mechanical sense. That gap between statistical prediction and real-world understanding is a limitation.

Second, missing values were handled simply, replace each one with the median of what was observed. That works, but it ignores the possibility that the missingness itself is informative. If a sensor stops reporting specifically when a component is under stress, that silence is a clue, not just a gap to fill in.

Third, each truck is treated as its own independent snapshot. There's no history, no sense of whether a truck's readings have been trending upward over the past month. Real failures often build up gradually, and a model that can see that progression would likely do better than one looking at a single moment in time.

Finally, the class imbalance was handled by reweighting during training. That works, but an alternative approach, actually creating more synthetic APS failure examples to train on, hasn't been tried and might give better results for some models, especially KNN.

5. Conclusion

5.1 What We Found

The goal of this project was simple: use sensor data from Scania trucks to predict whether a reported failure came from the Air Pressure System. The dataset made that harder than it sounds, nearly all failures in the data were non-APS failures, a lot of values were missing, and the cost of a missed APS failure was 50 times higher than the cost of a false alarm.

After cleaning the data, building six models, tuning each one, and comparing the results, the clearest finding was also the most surprising: the simplest model won. Logistic Regression ended up with the lowest real-world cost (23,900) because it caught the most actual failures, nearly 89% of them. XGBoost was more accurate and better calibrated by standard measures, but it was too conservative. It missed more failures, and in this problem, each missed failure hurts far more than a false alarm.

The data cleaning steps mattered just as much as the model choice. Adjusting for the class imbalance was essential. Without it, every model would have just predicted 'not an APS failure' for everything and missed all 375 real failures. Filling in missing values, removing nearly-empty features, scaling the data, and using PCA each played a role in making the models work properly.

5.2 Answers to the Four Questions

Q1: Can it be done? Yes. The best model cost 23,900 units on the test set. If we'd done nothing and missed every single APS failure, the cost would have been 187,500 units. So yes, the models find real patterns and save real money.

Q2: Which model is best? Logistic Regression wins on domain cost (23,900) because of its high recall. XGBoost wins on quality measures like F1 (0.822) and AUC (0.9946), and would likely win on cost too if its decision threshold were tuned. KNN is the weakest option for this kind of problem, it struggled with the rarity of APS failures in the training data and finished last.

Q3: Did the data cleaning matter? A lot. Class reweighting was non-negotiable, skip it and models ignore the rare APS failures entirely. Median imputation, feature filtering, scaling, and PCA all contributed to making the models work well. Skipping any of these steps would have hurt performance.

Q4: Which sensors matter most? A small group of features did most of the work. Random Forest and XGBoost largely agreed on which ones, a few histogram-style measurements tracking operating conditions showed up at the top of both lists. Most of the 167 features contributed little. A leaner model using just the top features would probably perform almost as well.

5.3 Ideas for Future Work

Four things stand out as natural next steps:

- Treat missing values as information. When a sensor stops reporting, that might not be random. It could be a sign something's wrong. Adding a simple yes/no column for each feature (was this value missing or not?) could give the model extra clues without needing any new data.

- Try generating synthetic failure examples (SMOTE). Instead of just telling the model that failures are important, SMOTE actually creates new synthetic APS failure examples by blending existing ones together. This gives the model more positive examples to learn from, which could particularly help KNN and other instance-based methods that struggle when the positive class is very rare.
- Tune the decision threshold. Every model in this study flagged a truck as an APS failure only when it was more than 50% confident. But given the 50:1 cost gap, it makes more sense to flag a truck even at 20% or 30% confidence. Finding the exact threshold that minimizes cost on a validation set rather than using the default 50%, could push XGBoost's domain cost well below Logistic Regression's, putting its superior AUC to better use.

Add time information. Right now, each truck is treated as a frozen snapshot. In the real world, sensors change over time as parts wear down, and the warning signs of an APS failure might show up days or weeks before the actual breakdown. If historical readings were available, a model that could spot a worsening trend would likely catch failures earlier and more reliably than any snapshot model can.

References

Get Started with XGBoost — *xgboost 3.2.0 documentation*.
(n.d.). https://xgboost.readthedocs.io/en/stable/get_started.html

RandomForestClassifier. (n.d.). Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

scikit-learn: machine learning in Python — *scikit-learn 1.8.0 documentation*.
(n.d.). <https://scikit-learn.org/stable/>

Shetty, R. (2025, March 5). *Predicting a Failure in Scania's Air Pressure System (APS)*. Towards Data Science. <https://towardsdatascience.com/predicting-a-failure-in-scanias-air-pressure-system-aps-c260bcc4d038/>

UCI Machine Learning Repository.
(n.d.). <https://archive.ics.uci.edu/dataset/414/ida2016challenge>

VanderPlas, J. (n.d.). *In-Depth: Support Vector Machines | Python Data Science Handbook*. <https://jakevdp.github.io/PythonDataScienceHandbook/05.07-support-vector-machines.html>

XGBoost Tutorials — *xgboost 3.2.0 documentation*.
(n.d.). <https://xgboost.readthedocs.io/en/stable/tutorials/>

Appendix A — Supplementary Tables and Notes

A.1 Detailed Preprocessing Pipeline Summary

Step	Method	Justification
Encode target	LabelEncoder: pos→1, neg→0	Required for scikit-learn estimators
Filter high-miss features	Drop features with >80% missing	Imputing >80% missing introduces excessive noise
Impute missing values	SimpleImputer(strategy='median')	Median is robust to skew and outliers
Scale features	StandardScaler (fit on train only)	Required for LR, KNN, SVM; prevents data leakage
Dimensionality reduction (LR, KNN, SVM)	PCA(n_components=n_95 %variance)	Removes multicollinearity; reduces curse of dimensionality
No reduction (DT, RF, XGB)	Full 167-feature matrix used	Tree models perform own feature selection; PCA not beneficial
Class imbalance correction	class_weight='balanced' / scale_pos_weight=59	59:1 imbalance makes uncorrected models biased to negative

Table A.1: Complete preprocessing pipeline with step-by-step justifications.